Reviar Architecture Guide

Reviar

Software Architecture & Requirements Specification (SRS)

Prepared by: Nash Dev (Oppong Nash Boateng) **Role:** Lead Systems Engineer & Mobile Developer *Revision 2 — reflects the shift from swipe-matching to browse-and-apply, the move from Sequelize to Prisma, native GitHub OAuth, RBAC-merged workspace membership, Kanban labels linked to channels, and the friends/DM system. Original SRS (CodeSync, swipe-based) superseded.*

---

1. Project Overview

1.1 Introduction

Reviar is a collaborative workspace platform tailored specifically for open-source software developers and startup founders. It bridges the gap between discovering technical talent and executing software projects by combining a searchable, GitHub-verified discovery feed with persistent, role-controlled communication channels — Discord-style workspaces that provision automatically the moment a Project Manager accepts a contributor.

1.2 Problem Statement

Finding collaborators for software projects is fragmented. Project Managers (PMs) post on static forums (Reddit, X), manually vet GitHub profiles, and manually invite and provision access to Discord servers. This context-switching creates friction, leading to abandoned projects and inefficient team building. Every existing “developer matchmaking” product stops at the introduction — none auto-provision the governed workspace that comes after.

1.3 Proposed Solution

A React Native (Expo) mobile app backed by a Node.js/Express API. Reviar surfaces open project roles in a searchable Explore feed, verifies technical skill via native GitHub OAuth, and — once a PM manually reviews and accepts an applicant — instantly provisions RBAC-gated workspace access to specific project channels.

Note on the matching model: the original swipe-based “Tinder for developers” mechanic was deliberately dropped. Every direct competitor in this space (HackMate, Match&Hack, CoffeeSpace, VenturLoop) already uses swipe cards, making it a category cliché rather than a differentiator. Reviar instead uses a scrollable, filterable Explore feed — cards are still visually interactive, but browsing and applying are separate, deliberate actions, and there is no mutual-match state. A PM reviews every application manually via a Requests page.

1.4 Vision & Mission

Vision: To become the universal infrastructure for spontaneous, high-efficiency software engineering teams.

Mission: Eliminate the friction of networking and context-switching, allowing developers to go from "discovery" to "deployment" in a single unified environment — and to own the moment *after* the introduction, which every competitor treats as the finish line.

1.5 Target Audience / User Personas

- **The Project Manager (PM) / Founder** — has a vision and core repository but needs specialists to scale. Reviews every application manually via the Requests page rather than relying on algorithmic matching.
- **The Open-Source Contributor** — a developer looking for real-world experience, browsing and applying to roles that fit their verified tech stack.
- **The Hackathon Sprinter** — needs to assemble a skilled team with precise technical requirements quickly, though Reviar's core positioning leans toward the weeks-to-months middle distance between hackathon urgency and long-haul co-founder search.

1.6 Scope

In Scope: Custom standalone authentication *plus* native GitHub OAuth (both as a login method and as an account-linking flow), GitHub profile ingestion (contribution graph, top languages, merged PR count), browse-and-apply project discovery with manual PM review, real-time WebSocket workspaces, Role-Based Access Control (RBAC) merged into workspace membership, Kanban boards with labels linked directly to channels, direct messaging (both within a shared workspace and via a standalone friends system), ephemeral audio Huddles.

Out of Scope (Phase 1): Built-in IDEs, direct code compilation within the app, payment gateways for freelance compensation, video conferencing (audio only).

---

2. Functional & Non-Functional Requirements

2.1 Core Features Overview

1. **Dual Authentication** — a standalone custom auth system (bcrypt + short-lived access JWT + HttpOnly-cookie refresh token) *and* native GitHub OAuth, which can either create/log into an account directly or link GitHub to an existing email/password account.
2. **Browse-and-Apply Discovery** — a searchable, filterable Explore feed of open project roles. No swiping, no algorithmic matching — applying is a deliberate, one-directional action a PM reviews manually.
3. **Integrated GitHub Portfolios** — contribution graph, top languages, public repo count, and merged PR count, fetched via Octokit and cached in a `GithubProfile` row rather than hit live on every profile view. Optionally gates a role's applicant pool via a minimum merged-PR threshold.
4. **Dynamic Workspaces** — auto-provisioned the moment a PM accepts an application, comprising multiple text channels (no separate persistent "voice channel" type — see 2.1.6).
5. **RBAC merged into workspace membership** — rather than a separate bridge table, each `WorkspaceMember` row carries its own `permitted_channels` array and an `is_pm` flag, since every member needs exactly one row regardless.
6. **Kanban with channel-linked labels** — task labels can optionally link directly to a channel, so tapping a label on a task jumps straight into the relevant channel's conversation.
7. **Ephemeral Huddles, not persistent voice channels** — audio calls are ad-hoc and loggable (who started it, when it ended) rather than a standing channel type. Live participant presence lives in Socket.io/Redis, not the database.
8. **Friends & direct messaging** — DMs are not limited to shared-workspace contacts. A standalone

`Friendship` model (request → accept, or block) governs who can DM whom outside of workspace context, while channel messages inside a shared workspace remain unaffected by friend/block status either way.

9. **Push notifications, not email, for product events** — matches, mentions, acceptances, and requests are delivered via Expo push + an in-app notification center. Email is reserved specifically for password reset, sent via the Gmail API.

2.2 Non-Functional Requirements

- **Scalability:** the Express backend supports horizontal scaling via stateless REST APIs and a Redis-backed Socket.io adapter for cross-process WebSocket state.
- **Real-Time Performance:** WebSocket latency should remain under 150ms for chat and Kanban synchronization.
- **Security:** passwords are hashed with bcrypt (salting is automatic and embedded in bcrypt's own output — no separate salt column). Refresh tokens are HttpOnly, `sameSite: strict` cookies; access tokens are short-lived (15 min) JWTs. `auth_token_version` on `User` allows instant, global invalidation of all outstanding tokens for a user (used on password reset and permission revocation). GitHub OAuth state is HMAC-signed and time-boxed to prevent CSRF on the callback.
- **Availability:** target 99.9% uptime on a cloud PaaS provider (Render/Railway).
- **Type Safety:** the entire backend is TypeScript, not JavaScript — chosen specifically because the frontend is already TypeScript (enabling shared type definitions), the schema is relationally complex enough that renamed/removed fields are a real risk, and GitHub API responses are messy enough to benefit from being typed once at the boundary.
- **Async style:** async/await with real `try/catch` throughout — no `.then()/.catch()` chains, no manual `Promise` construction except where wrapping a callback-based API leaves no alternative.

2.3 User Stories

- As a PM, I want to create a project listing with specific open roles so I can attract the right talent.
- As a Contributor, I want to filter the Explore feed by my tech stack so I only see relevant opportunities.
- As a PM, I want to review an applicant's GitHub stats within the app before accepting them.
- As a PM, I want to assign accepted contributors to specific sub-channels so they only access information relevant to their role.
- As a PM, I want a role to stay open to multiple accepted contributors rather than closing the moment one person is accepted, since this is team-building, not single-hire.
- As a Contributor, I want to keep talking directly to someone I met on a project even after that project ends, without needing to still share a workspace.
- As any user, I want to sign in with GitHub directly rather than creating a separate password if I don't want to.

---

3. System Architecture & Tech Stack

3.1 Technology Stack

Domain	Technology Chosen	Rationale
Frontend (Mobile)	React Native (Expo)	Cross-platform iOS/Android, rapid UI development, robust gesture libraries.
Backend API	Node.js & Express (TypeScript)	Familiar ecosystem, highly scalable, excellent async/real-time capabilities; TypeScript for shared types with the frontend and safety against schema drift.

Database (Core Relational)	PostgreSQL (Supabase) via **Prisma**	Crucial for RBAC, joins, and transactional integrity. Switched from Sequelize after repeated IPv4/pooler connection issues against Supabase; Prisma's `DATABASE_URL` (pooled) / `DIRECT_URL` (direct, for migrations) split resolved this natively.
Database (Chat Logs)	MongoDB (Mongoose)	Channel messages and DM threads — high-volume, unstructured payloads, referencing Postgres UUIDs as plain strings rather than enforced foreign keys.
Caching & Pub/Sub	Redis	WebSocket horizontal scaling via `@socket.io/redis-adapter`, and the cache layer for `permitted_channels` lookups on login.
Authentication	Standalone custom auth **+ native GitHub OAuth**	Full control over JWTs, refresh tokens, and session invalidation, with GitHub OAuth as an equally first-class login path — not a bolted-on afterthought.
Transactional Email	Gmail API (OAuth2)	Used exclusively for password reset. Every other notification (acceptance, mention, new request, friend request) goes through Expo push + the in-app notification center instead.
Real-time	Socket.io	Channel chat, Kanban sync, and Huddle presence signaling.
GitHub API client	Octokit	Powers both the OAuth user-info exchange and the contribution/language/PR-count stat fetching, cached into `GithubProfile`.

3.2 System Architecture Diagram (Textual Representation)

```
[Mobile Client: React Native + Expo]
 | (HTTPS / WSS)
 v
[Express API]
 |
 +---> [requireAuth] --> Validates access JWT (Bearer header)
 |
 +---> [REST API Services] (Express.js, TypeScript)
 | |--> Auth, GitHub OAuth, Projects, Applications, Workspaces,
 | |    Members/RBAC, Channels, Kanban, Friends, Notifications
 | |--> Reads/Writes --> [PostgreSQL via Prisma] (Supabase)
 |
 +---> [WebSocket Gateway] (Socket.io)
 |--> Authenticated via same access JWT (socket.handshake.auth.token)
 |--> Redis adapter for cross-process pub/sub
 |--> Channel messages + Huddle presence
 |--> Persists messages --> [MongoDB] (ChannelMessage / DmThread / DmMessage)
 |
 +---> [Gmail API] --> Password reset emails only
```

---

4. Database Design

Schema lives in `prisma/schema.prisma`, field names in snake_case matching the DB columns exactly (no `@map`).

4.1 PostgreSQL — Core Relational (Prisma)

Model	Purpose
`User`	Account, auth fields (`password_hash` nullable — GitHub-only accounts have none), `auth_token_version` for global token invalidation, cached `tech_stack` / `role_intent`, `password_reset_token_hash` + `password_reset_expires` for the stateless reset flow.
`GithubProfile`	1:1 cache of GitHub stats (`top_languages`, `contribution_graph`,

	`public_repos`, `merged_pr_count`, `last_synced_at`) — avoids hitting GitHub's API on every profile view.
`PushToken`	One row per device per user (a user can be logged in on multiple devices).
`Project`	A listing; `pm_id` is the **permanent** record of who created it.
`ProjectRole`	An open position. `status: open | closed` replaces an earlier `is_filled` boolean — since this is team-building, many people can be accepted into the same role, so "filled" as a binary never made sense. The PM closes a role deliberately, independent of headcount.
`Application`	One-directional apply record (`pending` / `accepted` / `declined`) — there is no "match" state; unique on `(project_role_id, applicant_id)` to stop duplicate applications.
`Workspace`	1:1 with `Project`, auto-created on... see note below.
`Channel`	Always text — no `type` enum. Unique per `(workspace_id, name)`.
`Huddle`	A **log**, not a channel type — `started_by`, `started_at`, `ended_at` (null = currently live). Live participant presence is Socket.io/Redis state, not this table.
`WorkspaceMember`	Membership **and** RBAC combined in one row — `is_pm`, `project_role_id` (nullable, since a PM doesn't fill their own listed role), and `permitted_channels` (JSONB array of channel ids), cached in Redis on login.
`TaskLabel`	Kanban label; `channel_id` optionally links it to a channel (nullable — not every label maps to one).
`Task` / `TaskAssignee`	Kanban card and its many-to-many assignees.
`Notification`	In-app notification list; `payload` JSON shape varies by `type`.
`Friendship`	New: directional friend request (`requester_id` → `addressee_id`), status `pending` / `accepted` / `declined` / `blocked`. Governs DM eligibility only — has no bearing on shared-workspace channel visibility.

Known schema gaps, intentionally left as application-layer concerns rather than DB constraints: - `Friendship`'s unique constraint only prevents duplicate requests in one direction (A→B); it does not stop B→A if A→B already exists. Must be checked in `friendController` before insert. - `Huddle.started_by` and `Task.created_by` were briefly inconsistent (one a bare string, one a proper relation) — both are now proper relations to `User`.

4.2 MongoDB — Chat (Mongoose)

Collection	Purpose
`ChannelMessage`	Workspace channel messages. `channelId` / `workspaceId` / `authorId` are Postgres UUID strings, not enforced FKs. Supports threading (`parentMessageId`) and pinning (`isPinned`).
`DmThread`	Exactly two `participantIds` — 1:1 only, per scope. Structurally independent of workspace membership, which is what makes "keep talking after the project ends" possible.
`DmMessage`	Individual DM messages, with `deliveredAt` / `readAt` timestamps to support a WhatsApp-style sent/delivered/read progression for a future dedicated DM interface.

---

5. API Design & Authentication Logic

5.1 Dual Authentication Strategy

Standalone custom auth, unchanged in spirit from the original design: - Registration/login: `bcrypt.compare` server-side; on success, issues an access token (15-min JWT, returned in the response body, held in memory client-side) and a refresh token (30-day JWT, HttpOnly/ `sameSite: strict` cookie). - `auth_token_version` on `User`: incremented on logout and on password reset, instantly invalidating every outstanding JWT for that user regardless of which device issued it. - **Known tradeoff:** refresh tokens are stateless (no server-side token

store/blacklist). A stolen refresh token remains cryptographically valid until either its own expiry or an `auth_token_version` bump — there's no per-token revocation. Acceptable for the current stage; a `RefreshToken` table with per-token `jti` tracking would be the upgrade path if stricter single-use rotation is ever needed.

Native GitHub OAuth, added since the original SRS: - Two modes, both signed into a single HMAC-protected `state` payload: `login` (no existing session — GitHub is the entire signup/login path) and `connect` (an already-authenticated user linking GitHub to their existing account). - State is signed with `crypto.createHmac('sha256', ...)`, base64url-encoded, verified with `crypto.timingSafeEqual` on callback — this, plus a short-lived (`10 min`) HttpOnly state cookie compared against the returned state, is what prevents CSRF on the OAuth callback. - On `login` mode with no matching account: a new `User` is created. If GitHub doesn't expose a verified primary email, a placeholder (`{githubId}-{username}@users.noreply.github.com`) is used rather than blocking signup — this is a deliberate tradeoff to keep GitHub-first signup frictionless, worth revisiting if email-dependent features (like password reset) need to apply to GitHub-only accounts later. - On `connect` mode: syncs GitHub stats onto the existing authenticated user, and explicitly rejects (409 Conflict) if that GitHub account is already linked to a *different* Reviar user. - GitHub stats (contribution graph, top languages, merged PR count) are fetched via Octokit and upserted into `GithubProfile` in the same transaction as the user resolution — not a separate follow-up call.

Password reset, via the Gmail API: - A random 32-byte token is generated; only its SHA-256 hash is stored (`password_reset_token_hash` + a 15-minute `password_reset_expires`), mirroring the "never store the raw secret" principle already used for passwords. - The raw token goes out in a reset-link email sent through the Gmail API (OAuth2-authenticated sender), not SMTP with a password — the only outbound email Reviar sends. - On successful reset, `auth_token_version` is incremented, which — as a side effect — logs out every other active session, not just the one that requested the reset. This is treated as a feature (a password reset should assume compromise and revoke everything), not a bug. - `forgotPassword` always returns the same success message whether or not the email exists, to avoid leaking which addresses have accounts.

5.2 Authorization Model: Ownership, Not Roles

There is no global "role" field gating access. The same person is a PM on their own project's workspace and a plain contributor on someone else's — access is checked per-resource:

Middleware	Checks
`requireAuth`	Valid access JWT present; attaches `req.userId`.
`requireProjectOwner` / `requireRoleParentProjectOwner` / `requireApplicationParentProjectOwner`	Walks up to `Project.pm_id`, from a project, role, or application id respectively.
`requireWorkspaceMember` / `requireWorkspacePM`	Checks `WorkspaceMember` existence / `is_pm` for `/workspaces/:id/...` routes.
`requireChannelParentWorkspacePM`	For `/channels/:id` routes, where only the channel id is in the URL — walks channel → workspace → `is_pm`.
`requireChannelAccess` / `requireChannelAccessByQuery`	Checks not just workspace membership but that the specific channel is in `permitted_channels` (or the requester is PM, who always has access) — the actual RBAC enforcement point for message routes.

5.3 Critical API Endpoints (representative, not exhaustive)

Method	Path	Notes
POST	`/api/v1/auth/register`	Standalone signup
POST	`/api/v1/auth/login`	Standalone login
GET	`/api/v1/auth/login/github`	Begin GitHub OAuth, `login` mode
GET	`/api/v1/auth/github`	Begin GitHub OAuth, `connect` mode (requires existing session)
GET	`/api/v1/auth/github/callback`	Shared callback for both modes
POST	`/api/v1/auth/refresh`	Rotates access + refresh tokens, checks `auth_token_version`
POST	`/api/v1/auth/logout`	Clears refresh cookie

POST	`/api/v1/auth/forgot-password` / `reset-password`	Gmail-delivered reset flow
GET	`/api/v1/projects/explore`	Browse-and-apply feed (search/filter/paginate)
POST	`/api/v1/roles/:roleId/applications`	Submit an application — replaces the old `/matches/apply`
GET	`/api/v1/projects/:id/applications`	PM's Requests page, grouped by role
PATCH	`/api/v1/applications/:id/accept`	PM accepts — triggers Grant Access, not a "match"
POST	`/api/v1/workspaces/:id/grant-access`	Creates the `WorkspaceMember` row with RBAC — replaces `/workspaces/grant`
PATCH	`/api/v1/workspaces/:id/members/:memberId/channels`	RBAC toggle
GET/POST	`/api/v1/workspaces/:id/tasks`, `/api/v1/workspaces/:id/labels`	Kanban + channel-linked labels
POST	`/api/v1/users/:id/friend-request`, `/api/v1/users/:id/block`	New: friends/DM system

Twelve route files in total (`auth` , `user` , `project` , `application` , `workspace` , `member` , `channel` , `message` , `dm` , `kanban` , `huddle` , `notification` , `friend`), all mounted flat at `/api/v1` rather than nested, since several resources are addressed both through a parent and directly by their own id.

---

6. Application Flow & Workflows

6.1 Browse, Apply, and Manual Review (replaces the old swipe-to-match flow)

```
(Contributor)               (Project Manager)             (System Backend)
     |                             |                             |
1. Scrolls Explore feed,           |                             |
   taps a card for detail          |                             |
     |                             |                             |
2. Taps "Apply to [role]" --------> |                             |
     |                        3. Sees new request on             |
     |                           Requests page (per-role tabs)    |
     |                             |                             |
     |                        4. Opens GitHub Profile Review      |
     |                           for the applicant                |
     |                             |                             |
     |                        5. Taps "Accept" ------------------> |
     |                                                      6. Creates WorkspaceMember
     |                                                         (RBAC via permitted_channels)
     | <------------------------------------------------------- 7. Fires push notification
8. Acceptance Modal fires                                        + Socket.io event
   on contributor's device
9. Taps "Go to workspace" -- lands directly in Workspace Home
```

Note there is no mutual-match state anywhere in this flow — acceptance is a unilateral PM decision, and the one celebratory UI moment left in the product (the Acceptance Modal) fires on the applicant's side at that decision point, not at apply-time.

6.2 GitHub OAuth (login mode)

```
[Mobile] --tap "Sign in with GitHub"--> GET /auth/login/github
     |
     v
Server signs HMAC state {mode: login, nonce, exp}, sets short-lived
state cookie, redirects to github.com/login/oauth/authorize
     |
     v
[GitHub] --user approves--> redirects to /auth/github/callback?code&state
     |
     v
Server: verify state cookie matches query state, verify HMAC signature,
```

```
exchange code for GitHub access token, fetch authenticated user + verified
email via Octokit, resolve/create Reviar User in a single transaction,
upsert GithubProfile with fetched stats, issue Reviar access+refresh tokens,
redirect to app deep link with access token in the URL fragment
```

---

7. Implementation Details & Structure

7.1 Folder / Project Structure

```
reviar_monorepo/
├── mobile/                    # React Native / Expo App (unchanged from original SRS)
│
├── backend/
│   ├── src/
│   │   ├── config/            # prisma.ts, mongo.ts, redis config
│   │   ├── controllers/       # 13 controllers — auth, user, project, application,
│   │   │                      # workspace, member, channel, message, dm, label,
│   │   │                      # task, huddle, notification, friend
│   │   ├── middlewares/       # requireAuth, requireProjectOwner (+2 variants),
│   │   │                      # requireWorkspacePM (+2 variants), requireChannelAccess
│   │   │                      # (+1 variant), asyncHandler, errorHandler
│   │   ├── routes/            # 13 route files, all mounted flat at /api/v1
│   │   ├── services/          # socketServer.ts, githubService.ts, emailService.ts
│   │   ├── utils/             # AppError.ts, jwt.ts, resetToken.ts
│   │   └── server.ts
│   ├── prisma/
│   │   └── schema.prisma
│   ├── tsconfig.json
│   └── package.json
```

7.2 Error Handling & Logging Strategy

- **API Layer:** a global `errorHandler` middleware, mounted last, returns `{ status: 'error', message, code }` for both thrown `AppError`s (expected conditions — 4xx) and unexpected exceptions (logged in full server-side, returned as a generic 500 to the client). An `asyncHandler` wrapper catches anything that escapes a controller's own `try/catch` and forwards it here, since Express 4 doesn't auto-catch rejected promises.
- **Code style:** async/await with real `try/catch` throughout — no `.then()/.catch()` chains anywhere in the codebase, adopted as a hard convention partway through the build.
- **Real-Time Layer:** Socket.io `connect_error` events surfaced to the React Native UI with auto-reconnect backoff.
- **Monitoring:** Sentry on both Express and React Native.

7.3 Package Management Note

Sequelize, `sequelize-cli`, and `pg` were removed entirely after the switch to Prisma — running both ORMs simultaneously against the same Supabase pooler was considered and explicitly rejected (doubles connection pool usage against a limited pgBouncer connection count, and reintroduces the exact IPv4/pooler issue that motivated the switch away from Sequelize in the first place).

---

8. Deployment & DevOps Roadmap

8.1 CI/CD Pipeline

```
[Developer Push] --> [GitHub Main Branch]
        |
   (GitHub Actions)
        |
   +----+----+
   |         |
[Run Jest Tests]   [Build RN Expo OTA Update]
   |                        |
[tsc build + Dockerize]     [Deploy via EAS]
   |
[Deploy to Render/Railway]
```

8.2 Development Phases

1. **Phase 1: Foundation** — monorepo, standalone auth, Prisma schema, basic Project CRUD. *(Complete — since revised to add GitHub OAuth ahead of schedule.)*
2. **Phase 2: Discovery Engine** — originally scoped as swipe UI; **revised** to the Explore feed + manual Requests review model instead.
3. **Phase 3: Real-Time Workspaces** — Socket.io, Redis, MongoDB chat schema, RBAC middleware.
4. **Phase 4: Polish & Deploy** — push notifications (Expo, not email), Dockerize, deploy, end-to-end testing.

---

9. Risks & Mitigation

Risk	Impact	Mitigation Strategy
WebSocket connection drain	High	Redis Pub/Sub via `@socket.io/redis-adapter` for horizontal scaling.
Custom Auth vulnerability	Critical	Short-lived access JWTs, HttpOnly refresh cookies, explicit `auth_token_version` invalidation on password reset/role changes.
Database query bottleneck	Medium	Indexes on `user_id` / `workspace_id` -style foreign keys throughout; `permitted_channels` cached in Redis on login.
GitHub OAuth CSRF	High	HMAC-signed, time-boxed `state` parameter, cross-checked against a short-lived HttpOnly state cookie, verified with constant-time comparison.
Refresh token replay	Medium	No per-token blacklist currently exists — mitigated only by `auth_token_version` bumps on logout/reset, not per-token revocation. Documented tradeoff; upgrade path is a `RefreshToken` table with `jti` tracking if needed later.
Dual-ORM drift	Medium	Resolved by fully removing Sequelize rather than running it alongside Prisma for any subset of routes.
Blocked-user leakage	Low	`Friendship.status = 'blocked'` is checked only on the DM creation/send path — explicitly does *not* apply to shared workspace channels, which was a deliberate product decision, not an oversight.